

Decisões de Arquitetura

Como o sistema está organizado e por que foi feito assim

Para quem é este documento. Equipe técnica que desenvolve, implanta ou mantém o GeopetrolO. Ele resume as decisões que dão forma ao sistema e o motivo de cada uma.

Como ler. A parte 1 mostra o conjunto e o fluxo de dados. A parte 2 traz cada decisão no formato contexto, decisão e consequências. A parte 3 lista riscos aceitos e pendências. Os detalhes vivem nas specs do repositório, indicadas em cada decisão.

Situação. Em outubro de 2026 o sistema ainda não está em produção. Por isso algumas mudanças entraram sem migração gradual nem compatibilidade com nomes antigos.

Sumário

1. Visão geral	3
Fluxo de arquitetura	4
Dentro do Desktop	5
2. Decisões	6
D01 · Monorepo com aplicações independentes	6
D02 · Braserv-Core é dono da identidade e do cadastro organizacional	6
D03 · Só o Core emite tokens, assinados com RS256	6
D04 · Uma origem única pelo proxy	7
D05 · Dois caminhos independentes para a telemetria	7
D06 · MQTT em lote com mensagem que se descreve	7
D07 · Serviço de Telemetria com InfluxDB	7
D08 · Como a unidade é identificada	8
D09 · Desktop como agente de borda dirigido por configuração	8
D10 · Identidade estável do card	8
D11 · Calibração fica na estação	8
D12 · O servidor registra o alarme e a estação sinaliza	9
D13 · Configuração restrita, com sessão só em memória	9
D14 · Implantação separada por caminho de dados	9
D15 · Arquitetura hexagonal por módulo	9
3. Riscos aceitos e pendências	10
Onde aprofundar	10

1. Visão geral

O GeopetroIO coleta dados de unidades em campo, como sondas, unidades de bombeio e cimentação, e os entrega à supervisão remota e ao cliente. São seis aplicações independentes.

Aplicação	Tecnologia	Papel
Braserv-Core	Spring Boot · Java 21	Identidade, login, tokens e cadastro organizacional: usuários, empresas, regionais, setores e unidades.
Geopetro-Backend	Spring Boot · Java 21	Monitoramento, tempo real, alarmes, configuração de cards e simulador de cimentação.
Geopetro-Front	Angular 21 · Taiga UI	Aplicação web para cadastro, monitoramento, alarmes, simuladores e relatórios.
Geopetro-Desktop	JavaFX 21 · Spring Boot	Agente de borda no PC da unidade. Lê o CLP LOGO!, mostra o painel local e publica telemetria.
Geopetro-Telemetria	Spring Boot · InfluxDB	Consome o MQTT, grava as séries no InfluxDB e responde consultas de histórico ao Backend.
Braserv-Horus	JavaFX 21 · Gradle	Aplicação de cimentação autônoma. Lê o CLP da bomba e gera a carta de operação em PDF.

Fluxo de arquitetura

O diagrama mostra a implantação em duas máquinas virtuais. Os números nas setas correspondem à tabela logo abaixo.

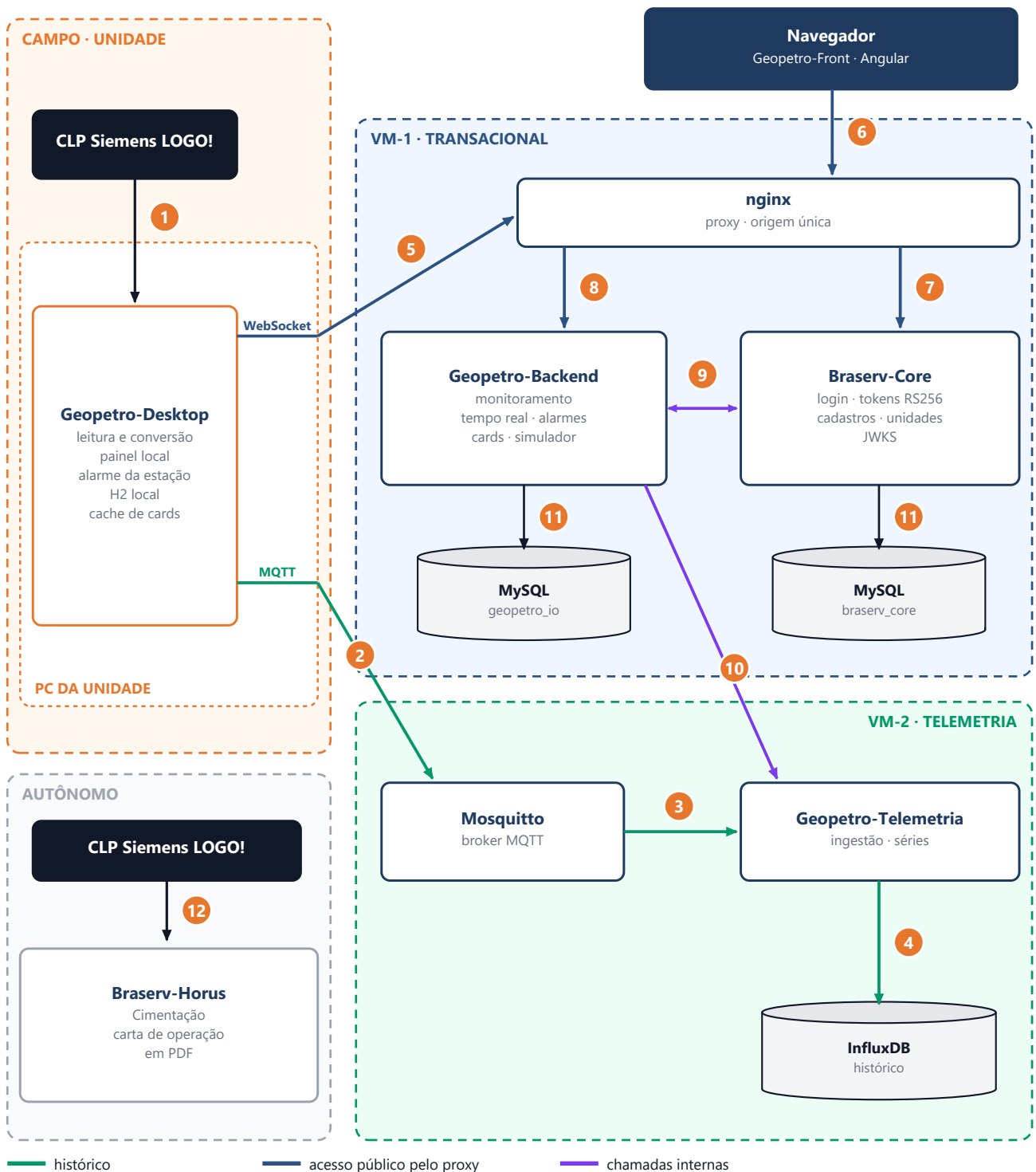


Figura 1 — Fluxo de arquitetura do GeopetroIO.

Nº	Origem e destino	Protocolo	Para quê
1	CLP LOGO! → PC da unidade	S7 · TCP 102	O Geopetro-Desktop, no PC da unidade, lê a memória V do LOGO! a cada ciclo, normalmente 1 s, e converte cada valor conforme o tipo do card.
2	PC → Mosquitto	MQTT · QoS 1	Histórico. Uma mensagem por ciclo no tópico telemetria/{idUnidade}/batch.
3	Mosquitto → Telemetria	MQTT	Assinatura de telemetria/+/batch. Um único consumidor.
4	Telemetria → InfluxDB	HTTP · lote	Gravação assíncrona em lotes de 500 pontos ou 1 s, em UTC.
5	PC → nginx	HTTPS · WebSocket	Login no Core, envio do tempo real por STOMP e recebimento do documento de cards.
6	Navegador → nginx	HTTPS · WSS	Telas web e assinatura do tempo real.
7	nginx → Core	HTTP	Login, usuários, empresas, regionais, setores, unidades e clientes de serviço.
8	nginx → Backend	HTTP · STOMP	Monitoramento, alarmes, simulador e /ws.
9	Backend ↔ Core	HTTP interno	Chaves JWKS, acesso atual do usuário, catálogo de unidades e vínculos antes de excluir.
10	Backend → Telemetria	HTTP interno	Consulta de séries históricas. O Front nunca chama a Telemetria.
11	Core e Backend → MySQL	JDBC	Mesmo servidor MySQL, databases e usuários separados.
12	CLP LOGO! → Horus	S7 · TCP 102	Cimentação autônoma, sem integração com as demais aplicações.

Dentro do Desktop

Uma thread virtual lê o LOGO!, converte os valores e entrega a leitura para quatro destinos. Nenhum destino bloqueia a leitura nem os outros destinos.

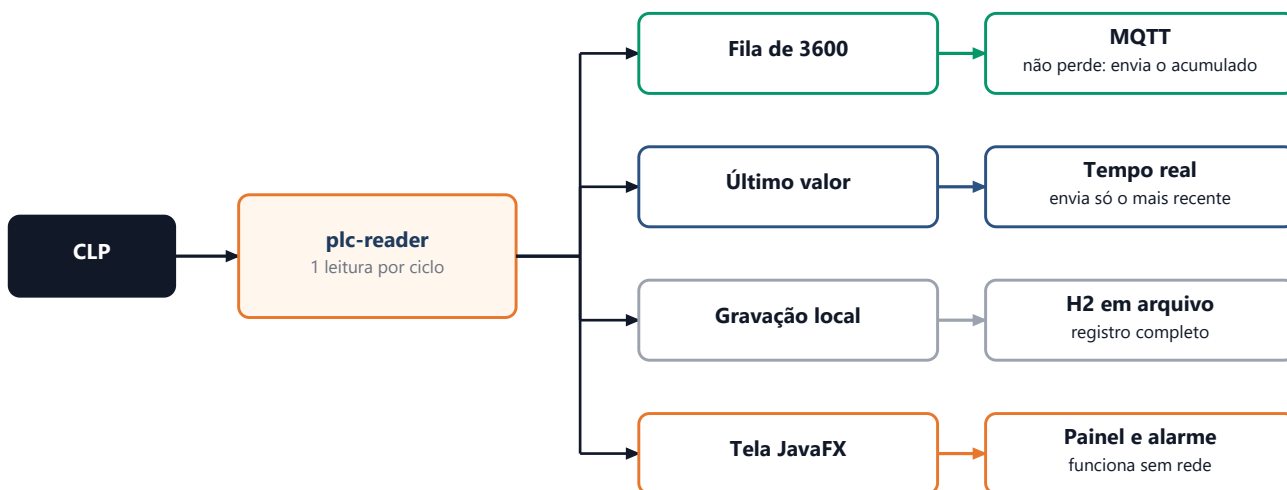


Figura 2 — Como a leitura se divide dentro do Geopetro-Desktop.

2. Decisões

Cada decisão traz o problema que a motivou, o que foi decidido e o que isso provoca no sistema. As referências apontam para as specs em specs/SDD.

D01 · Monorepo com aplicações independentes

Contexto	O sistema reúne coleta em campo, monitoramento web, histórico e simulação. Cada parte tem ritmo de entrega e forma de implantação diferentes.
Decisão	Seis aplicações implantáveis em apps/, cada uma com build e Dockerfile próprios. O Braserv-Horus continua separado e não será absorvido pelo Geopetro-Desktop.
Consequências	• Cada aplicação é construída e implantada sozinha. • A duplicação entre os dois desktops é aceita de propósito (DT-010). • Quatro classes de base são copiadas entre Core e Backend. Uma biblioteca comum só será extraída quando um terceiro serviço precisar.
Referências	README.md · software/system-overview.md · software/technical-debt.md

D02 · Braserv-Core é dono da identidade e do cadastro organizacional

Contexto	Usuário, empresa, regional, setor e unidade servem a outros sistemas da Braserv, não só ao GeopetroIO.
Decisão	Esses cadastros e o login pertencem ao Braserv-Core. O Backend não copia tabelas e lê pelo contrato interno do Core. Cada serviço tem database e usuário próprios no mesmo servidor MySQL: braserv_core e geopetro_io.
Consequências	• Um sistema novo reaproveita login e estrutura sem depender do Backend. • No Backend, unidade_id é referência lógica, sem chave estrangeira entre databases. • Excluir uma unidade exige confirmação do Backend e da Telemetria. Sem resposta em 5 s, a exclusão é recusada. Unidade já usada é inativada, nunca excluída.
Referências	software/backend/braserv-core.md · RN-115 a RN-119

D03 · Só o Core emite tokens, assinados com RS256

Contexto	Vários serviços precisam confiar na identidade do usuário sem compartilhar um segredo.
Decisão	O Core assina com a chave privada e publica as chaves públicas no JWKS. Token de pessoa vale 1 h e token de serviço vale 15 min. O campo tipo impede que token de pessoa abra rota interna e que token de serviço abra rota pública. O Backend consulta no Core o acesso atual do usuário, com cache de 10 s.
Consequências	• Nenhum consumidor guarda segredo de assinatura. • Desativar um usuário corta o acesso em até 10 s. • Com o Core fora do ar, o Backend usa o último acesso conhecido por até 5 min e depois nega. É o custo aceito para que um reinício do Core não derrube o tempo real. • A revogação individual de token continua pendente (SEC-008).
Referências	software/backend/braserv-core.md §6 · software/apis/braserv-core.md

D04 · Uma origem única pelo proxy

Contexto	O navegador fala com dois serviços, e as rotas internas não podem ficar expostas.
Decisão	O nginx do Front publica uma única origem. Identidade e cadastros vão ao Core. Monitoramento, simulador e /ws vão ao Backend. As rotas /internal/** e /.well-known/** não são publicadas. O Front nunca chama a Telemetria.
Consequências	• Não há CORS entre serviços no navegador. • A superfície pública fica menor. • O Backend valida o acesso à unidade antes de consultar a Telemetria.
Referências	software/system-overview.md §3

D05 · Dois caminhos independentes para a telemetria

Contexto	O histórico não pode perder leituras. O tempo real só precisa do estado mais recente.
Decisão	MQTT leva o histórico até a Telemetria e o InfluxDB. WebSocket com STOMP leva o estado atual ao Backend, que o repassa ao Front sem gravar. No Desktop, o MQTT usa uma fila de 3600 leituras e o tempo real guarda só o último valor. Cada caminho tem seu próprio worker em thread virtual.
Consequências	• A falha de um caminho não afeta o outro nem a leitura do CLP. • A fila cobre cerca de uma hora de broker fora do ar. O banco H2 local guarda o registro completo. • Ao reconectar, o tempo real envia só o estado mais recente. • Cada caminho pode ser desligado na engrenagem da estação.
Referências	software/apis/websocket-realtime.md · software/mqtt/mqtt-telemetria.md

D06 · MQTT em lote com mensagem que se descreve

Contexto	Uma mensagem por sensor multiplicaria o volume, e brokers gerenciados cobram por mensagem.
Decisão	Uma mensagem por ciclo, QoS 1, no tópico telemetria/{idUnidade}/batch. Cada leitura carrega tipo, série e endereço, então não existe catálogo central. Só o Desktop publica e só a Telemetria consome. O Backend saiu do broker.
Consequências	• O volume é cerca de 5 vezes menor que o formato por sensor. • Leitura sem valor, como peso ainda sem calibração, não é publicada. • Pendentes: autenticação no broker (SEC-009) e buffer de contingência no produtor.
Referências	software/mqtt/mqtt-telemetria.md

D07 · Serviço de Telemetria com InfluxDB

Contexto	A escrita é contínua, 24 horas por dia, e as consultas pedem horas de dados a cada 1 s.
Decisão	Um serviço próprio grava as séries no InfluxDB em lotes assíncronos e sempre em UTC. Consultas acima de 2000 pontos por série são reduzidas. O broker fora do ar não impede o serviço de subir.
Consequências	• A carga de escrita fica separada do MySQL transacional. • A tela de histórico recebe volume controlado mesmo em janelas longas. • A Telemetria não conhece usuários. Quem controla acesso é o Backend.
Referências	apps/geopetro-telemetria/specs · software/apis/rest-monitoramento.md

D08 · Como a unidade é identificada

Contexto	A web precisa de um identificador estável. O contrato de coleta já usava o nome da unidade.
Decisão	O cadastro se chama Unidade em todo o sistema. Web, REST e WebSocket usam o id numérico. MQTT e InfluxDB usam o nome da unidade como chave. O Backend traduz um no outro antes de consultar a Telemetria.
Consequências	• O Front trabalha com um único identificador. • Renomear uma unidade corta a série histórica em duas. É o risco de maior custo registrado (RN-018).
Referências	RN-018 · RN-119 · <code>software/backend/braserv-core.md</code> §9

D09 · Desktop como agente de borda dirigido por configuração

Contexto	Cinco grandezas fixas no código impediam instrumentar equipamentos que não fossem sonda.
Decisão	Cada unidade tem um documento de cards com a conexão do CLP e a lista de cards, cada um com tipo e endereço. Ele é editado só no Desktop, por ADMIN ou SUPORTE, guardado no Backend com número de revisão, entregue pelo canal STOMP /cards e mantido em cache em disco. Há seis tipos fechados. A leitura é feita em bloco, uma faixa do DB por ciclo.
Consequências	• Instrumentar um equipamento novo vira cadastro, não projeto. • Unidade sem cards não lê nem publica (RN-088). • Duas edições simultâneas são recusadas pela revisão, com erro 409. • Ao copiar de outra unidade, o IP do CLP não é copiado.
Referências	<code>negocio/requisitos/cards-configuraveis.md</code> · <code>software/apis/configuracao-sonda.md</code>

D10 · Identidade estável do card

Contexto	O id do card vira tag no InfluxDB, com retenção de 5 anos. Tag com valores livres degrada o banco.
Decisão	O id é gerado no formato TIPO_NN, nunca muda e não é reaproveitado. O nome é só rótulo de tela. Card não se exclui, se desativa. O limite é de 100 cards por unidade.
Consequências	• A quantidade de tags no InfluxDB fica sob controle. • Toda série antiga continua tendo um card que explica o que ela mede.
Referências	RN-081 · RN-091 · <code>software/mqtt/mqtt-telemetria.md</code> §4

D11 · Calibração fica na estação

Contexto	Geometria de peso e de chave hidráulica são medidas em campo, na própria unidade.
Decisão	A calibração de peso e torque fica no computador da unidade, em <code>config/calibracao-cards.json</code> , separada pelo id do card. O range do transmissor fica no documento da unidade.
Consequências	• Os valores medidos em campo não trafegam pela rede. • Trocar o computador exige refazer ou copiar a calibração.
Referências	<code>negocio/requisitos/cards-configuraveis.md</code> §10b

D12 · O servidor registra o alarme e a estação sinaliza

Contexto	Supervisão e cliente precisam de um histórico único. A equipe na unidade precisa de alarme mesmo sem rede.
Decisão	O Backend é o único produtor do histórico de alarmes. Ele avalia os limites remotos sobre o canal de tempo real. O alarme é a única parte do sistema orientada a eventos: cada episódio é uma sequência só de inclusões. A estação tem limites locais próprios, funciona sem rede e sem login, e não gera evento.
Consequências	• Existe uma única fonte de verdade para o histórico de alarmes. • Unidade com tempo real caído, ou card invisível, não alarma no servidor mesmo publicando MQTT. • O alarme local só chega à frota com o auto-update do Desktop, que ainda não existe.
Referências	negocio/requisitos/alarmes.md · negocio/requisitos/configuracao-da-estacao.md

D13 · Configuração restrita, com sessão só em memória

Contexto	Acertar endereço, rack e slot exige estar na unidade. Existe uma credencial de serviço comum à frota.
Decisão	Configurar exige ADMIN ou SUPORTE autenticado online. A sessão vive em memória e não é gravada em disco. Sem login, o Desktop continua lendo, publicando e alarmando.
Consequências	• Nenhuma credencial extra fica gravada na borda (SEC-011). • Configurar uma estação exige internet.
Referências	RN-086 · negocio/requisitos/configuracao-da-estacao.md §5

D14 · Implantação separada por caminho de dados

Contexto	A ingestão escreve sem parar e recebe conexões das unidades. A parte web é intermitente e guarda o cadastro de usuários.
Decisão	Duas máquinas virtuais. A transacional tem Front, Core, Backend e MySQL. A de telemetria tem Mosquitto, Telemetria e InfluxDB. Existe também uma alternativa em máquina única, com Caddy e TLS automático. Os bancos MySQL são versionados com Flyway e validados na subida.
Consequências	• Uma falha na web não interrompe a ingestão. • Cada lado escala de forma independente. • O InfluxDB fica junto do consumidor, então a escrita contínua não atravessa a rede. Entre as máquinas só passa a consulta REST.
Referências	deploy/README.md · deploy/vm-unica/README.md

D15 · Arquitetura hexagonal por módulo

Contexto	Core e Backend têm vários domínios e dependem um do outro por rede.
Decisão	Core e Backend são projetos Maven com um módulo por domínio, organizados em portas e adaptadores. A integração entre serviços passa por portas, como <code>AcessoDoUsuarioPort</code> e <code>CatalogoDeUnidadesPort</code> .
Consequências	• Trocar um adaptador, como o cliente HTTP do Core, não toca a regra de negócio. • Os testes de domínio rodam sem rede.
Referências	software/backend/braserv-core.md §7 e §10

3. Riscos aceitos e pendências

Estes pontos são conhecidos e foram aceitos ou estão aguardando decisão.

Ponto	Efeito	Situação
Broker STOMP em memória	Com duas instâncias do Backend, o tempo real não passa de uma para a outra.	Aceito enquanto houver uma instância. Solução prevista: broker externo.
Broker MQTT sem autenticação	Qualquer cliente na rede pode publicar no tópico de telemetria.	Pendente (SEC-009).
Nome da unidade como chave	Renomear corta a série histórica.	Risco aceito (RN-018).
Sem auto-update do Desktop	Correções só chegam às unidades por visita.	Pendente.
Buffer de contingência MQTT	Broker fora por mais de uma hora descarta as leituras mais antigas da fila.	Pendente (P-07). O H2 local mantém o registro.
Token sem revogação individual	Um token vazado vale até expirar.	Pendente (SEC-008).
Repositórios dentro do OneDrive	A sincronização já corrompeu um repositório Git.	Pendente (DT-004).

Onde aprofundar

Assunto	Documento
Visão do sistema	specs/SDD/software/system-overview.md
Braserv-Core	specs/SDD/software/backend/braserv-core.md
Tempo real	specs/SDD/software/apis/websocket-realtime.md
Telemetria MQTT	specs/SDD/software/mqtt/mqtt-telemetria.md
Cards configuráveis	specs/SDD/negocio/requisitos/cards-configuraveis.md
Alarmes	specs/SDD/negocio/requisitos/alarmes.md
Implantação	deploy/README.md
Regras e pendências	specs/SDD/negocio/regras/business-rules.md · requisitos/open-questions.md